

Auto-Review Agent

Team Name: Team 9

Team Members:

Shayan Asif
Luis Jaimes
Hamere Abate
Adith Gangalakunta
Carlos Ortiz III

Supervisor:

Professor Sridhar Alagar

Project Coordinator:

Adarsh Gella

**Erik Jonsson School of Engineering and Computer Science
University of Texas at Dallas**

Date: 9 May 2026

Table of Contents

Executive summary

1. Introduction

2. Project Specification & Management

3. System Architecture & High-Level Design

4. Implementation & AI Orchestration

5. System Performance & Validation 6.

Lessons Learned

7. Conclusion & Future Work

- **References**

- **Acknowledgements**

Executive summary

The Intelligent Email Approval Automation System is an AI-powered workflow automation platform designed to streamline how organizations process approval-based requests submitted through email or in-app forms. Traditional approval systems often rely on manual review, which leads to delays, repetitive administrative work, and inconsistent decision-making. Our system addresses this issue by combining AI analysis, workflow automation, database management, and a modern frontend dashboard into a single integrated solution.

The system automatically reads incoming requests, classifies the type of request, evaluates the request using AI and rule-based logic, stores the results in a centralized database, and sends automated responses when appropriate. Low-risk requests, such as room booking inquiries, can be automatically approved, while higher-risk or ambiguous requests are escalated for human review. The platform also includes support for asynchronous workflow execution, scalable API architecture, and multi-user request handling.

The project integrates several technologies including React, FastAPI, Supabase, n8n, and large language models such as Gemini and Groq. Throughout development, the team focused on automation, scalability, workflow orchestration, and AI-assisted decision-making. The final product demonstrates how modern AI systems can reduce manual workload, improve response time, and support scalable approval automation for real-world organizational environments.

Introduction

1.1 Motivation & Problem Statement: Many organizations rely heavily on email communication for approval-based processes such as room reservations, access requests, asset requests, and scheduling inquiries. These requests are typically reviewed manually by administrators or staff members, creating several operational inefficiencies:

- Slow response times
- Repetitive manual review
- Inconsistent approval decisions
- Difficulty managing high request volume
- Lack of centralized tracking and visibility

As organizations grow, handling these requests manually becomes increasingly inefficient and difficult to scale. Delays in processing can negatively impact productivity, while repetitive administrative tasks consume valuable time that could otherwise be spent on higher-priority work.

The motivation behind this project was to design a system capable of intelligently automating these workflows while still allowing human oversight when necessary. The team wanted to explore how AI models, workflow automation tools, and cloud-based infrastructure could work together to reduce administrative overhead and improve operational efficiency.

Additionally, the project aimed to demonstrate how AI can be integrated into practical business workflows rather than used solely for conversational applications. By automating low-risk approvals and escalating higher-risk cases, the system creates a balanced approach between automation and human decision-making.

1.2 Objectives, Scope, and Goals Achieved:

Objectives

The primary objectives of the project were:

- Automatically process incoming request emails
- Use AI to classify and evaluate requests
- Route requests through different processing paths
- Automatically approve low-risk requests

- Escalate complex or high-risk requests for human review
- Store requests and decisions in a centralized database
- Build a frontend dashboard for monitoring requests
- Support scalability for multiple concurrent users and requests
- Demonstrate asynchronous workflow automation

Scope

The scope of the project focused on creating a functional end-to-end approval automation system with the following components:

- Email processing workflow
- AI-based request classification
- Workflow automation using n8n
- Backend API services using FastAPI
- Database storage using Supabase
- Frontend dashboard for request visualization
- Automated email response generation
- Basic scalability and monitoring concepts

The project did not aim to fully replace enterprise-grade approval systems or implement advanced security/compliance infrastructure. Instead, the focus was on building a realistic proof-of-concept demonstrating automation, AI integration, and scalable system design principles.

Goals Achieved

During development, the team successfully achieved several major engineering goals:

- Built a working automated email processing pipeline
- Integrated AI models for request analysis and recommendation generation
- Developed request routing logic for different request categories
- Connected frontend, backend, workflows, and database systems together
- Implemented automated email responses
- Created a centralized database structure for request tracking
- Improved workflow scalability by processing multiple unread emails efficiently
- Integrated asynchronous workflow execution concepts
- Deployed frontend and backend infrastructure online
- Demonstrated scalable system design strategies for high request volume environments

1.3 Proposed Solution:

The proposed solution was an integrated AI-powered approval automation platform capable of processing organizational requests with minimal manual intervention.

The system operates through several connected components working together as a unified workflow:

1. Incoming emails are detected through the automation workflow.
2. The workflow extracts important information such as sender, subject, and request content.
3. AI models analyze the request and generate structured outputs including summaries, recommendations, and classifications.
4. Requests are routed into different processing paths depending on request type and risk level.
5. Low-risk requests may be automatically approved using predefined logic and database checks.
6. Higher-risk or uncertain requests are escalated for human review.
7. Processed results are stored in the Supabase database for tracking and auditing.
8. Automated responses are generated and sent back to users.
9. The frontend dashboard displays requests, statuses, and asset information for administrators.

The solution combines AI-assisted reasoning with rule-based workflows to create a balanced and scalable approval system. By integrating asynchronous processing, centralized storage, and modular APIs, the architecture supports future scalability and expansion into additional approval scenarios beyond room booking and asset management.

1.4 Key Highlights:AI Integration

- Integrated large language models (Gemini and Groq) for natural language understanding
- Generated AI summaries and recommendations from unstructured email content
- Implemented automated approval and escalation logic

Workflow Automation (n8n)

- Built asynchronous workflows for processing incoming requests
- Implemented request routing logic for different request categories
- Added automated email response functionality

- Improved workflow throughput by processing multiple unread emails more efficiently

Backend Development (FastAPI)

- Developed lightweight API endpoints for frontend and database communication
- Structured backend services to support scalability and modularity
- Enabled centralized access to processed request data

Database Integration (Supabase)

- Designed structured database tables for emails, requests, and assets
- Enabled persistent tracking of approvals and workflow results
- Supported room and asset availability management

Frontend Development

- Built a React-based dashboard for visualizing requests and approvals
- Connected frontend components to live backend data
- Improved UI design and usability for administrators

Scalability & System Design

- Designed the system with asynchronous processing concepts
- Explored queue-based workflow execution and multi-request handling
- Implemented concepts such as caching, rate limiting, and monitoring strategies
- Investigated scalability improvements for handling large request volumes

Key Findings

- AI-assisted workflows significantly reduce repetitive manual review tasks
- Workflow orchestration tools like n8n simplify automation integration
- Request routing and selective AI usage improve efficiency and reduce unnecessary API usage
- Scalability challenges primarily involve workflow concurrency and AI processing limits rather than frontend limitations

Project Specification & Management

2.1 The Project Specification (The AI's Blueprint + Link): The project specification

document served as the central blueprint for the Intelligent Email Approval Automation System. The specification was designed as an “agent-ready” technical document containing detailed descriptions of the system architecture, workflows, frontend requirements, backend functionality, database structure, AI processing logic, and scalability considerations. The document acted as the primary reference point for both team members and AI-assisted development tools throughout the project lifecycle.

The specification included:

- Functional requirements for email processing
- Frontend dashboard requirements
- Backend API behavior
- Database schema concepts
- Workflow automation logic using n8n
- AI decision-making flow
- Scalability and deployment planning
- Future improvements and feature ideas

The document was continuously updated as the project evolved to ensure the implementation remained aligned with the latest design decisions and technical goals.

Project Specification Link:

Main Specification Document: [W Specifications Document.docx](#)

2.2 Maintaining the "Single Source of Truth": To maintain consistency across development, the team treated the specification document and GitHub repository as the “single source of truth” for project requirements and implementation progress. This ensured that all members were working from the same understanding of the system architecture and feature expectations.

Version Control Process

- GitHub was used for source control and collaboration.
- Team members pushed updates through commits and pull requests.
- Frontend and backend updates were synchronized through the shared repository.
- Shared documentation folders were used for reports, proposals, and project planning materials.

Documentation Management

The specification document was updated whenever:

- New features were added
- Workflow logic changed
- Scalability requirements evolved
- Deployment decisions were finalized
- API/database structures were modified

This process reduced confusion and prevented team members from implementing outdated logic or conflicting system behaviors.

AI Alignment

Since the project involved extensive AI-assisted development, keeping the documentation updated was especially important. The specification acted as a structured instruction set describing:

- desired behaviors,
- workflow expectations,
- routing logic,
- scalability goals,
- and system architecture decisions.

Maintaining updated documentation helped ensure AI-generated code and workflow suggestions remained aligned with the latest project requirements.

2.3 Key Milestones:

Initial Planning & Problem Definition

The first major milestone involved identifying the core problem the team wanted to solve: reducing manual approval workload through AI-assisted automation. During this phase, the team explored possible use cases including room bookings, asset requests, and access approvals.

Technical Specification Completion

After defining the problem space, the team created the initial technical specification document outlining:

- system architecture,
- workflow structure,
- frontend/backend responsibilities,
- AI integration goals,
- and scalability considerations.

This milestone established the foundation for development.

Minimum Viable Product (MVP)

The first functional MVP successfully demonstrated:

- fetching unread emails,
- processing them through n8n workflows,
- generating AI-based analysis,
- and storing results in the database.

This represented the first complete end-to-end automation cycle.

Frontend Integration

The next milestone involved integrating the React frontend with backend APIs and live Supabase data. Dashboard components were connected to real request information, allowing administrators to monitor requests visually.

AI Workflow Expansion

The workflow system was expanded to support:

- request routing,
- multiple request categories,
- automated responses,
- and escalation logic for human review.

Additional work focused on improving processing efficiency and reducing unnecessary AI calls.

Deployment & Online Hosting

The team deployed the frontend and backend systems online using Vercel and cloud-hosted infrastructure. This allowed team members to collaborate remotely and demonstrate the project outside of local development environments.

Scalability Improvements

Later milestones focused on scalability concepts including:

- asynchronous workflow execution,
- handling multiple unread emails,
- reducing API bottlenecks,
- and exploring parallel request processing using faster AI models such as Groq.

2.4 Team Roles & Leader Rotation: A record of the team leader rotation schedule and a specific breakdown of the technical responsibilities assigned to each member throughout the project.

Week 1, 2 Shayan Asif - organized meetings, designed prototype for UI, helped team set up AI tools such as Codex and Claude, managed frontend engineering

Week 3, 4 Carlos Ortiz III - Team leader for Week 4; lead manager of Jira board, lead for bug reports, QA, helped with various documentation & Specifications Document

Week 5 Adith Gangalakunta - led the team while responsibilities were divided across backend AI workflow integration and Supabase/frontend connection, frontend UI exploration and prototyping, email automation through Zapier and n8n, and project documentation/Jira tracking. The team mainly focused on building the backend pipeline, connecting systems locally, and advancing the project specification

Week 6 -8 Adith Gangalakunta - continued as team lead while the team split work across deployment and repo consolidation, dashboard and landing page design, risk/specification planning, Vercel and Supabase setup, and testing the live web app workflow. The main focus shifted toward polishing the frontend, deploying the system online, and improving end-to-end integration reliability.

Week 9 - 10 Hamere Abate - helped team implement scalability initiatives while reducing costs, worked on risk management and worked on fixing n8n workflow to allow Gmail responses

Week 11 - 12 Shayan Asif - started preparing project for hosting and CI/CD pipeline, worked on bug fixes, made professional designs for UI pages

Week 12 - 16 Luis Jaimes- created the final workflow that the final product will use.

With this, I updated the Supabase to reflect and function alongside the n8n workflow. Everyone else worked on final touches to the frontend and added extra functionalities to the frontend, such as adding comments to people's requests. Also, they worked on bug reports to put a spotlight on what needs attention. Overall, just getting prepared for the final demo and getting everything tidied up.

System Architecture & High-Level Design

3.1 System Overview:

3.1 System Overview

The Intelligent Email Approval Automation System was designed as a modular, AI-assisted workflow platform capable of automating organizational request handling. The system integrates frontend interfaces, backend APIs, workflow orchestration tools, AI processing services, and cloud-hosted databases into a unified architecture.

The primary goal of the architecture was to support:

- automated email processing,
- AI-based decision making,
- centralized request tracking,
- asynchronous workflow execution,
- and scalable multi-user request handling.

The system processes incoming requests through a sequence of connected services that work together to automate approvals while still allowing human oversight when needed.

Core System Components

Frontend (React + Vercel)

The frontend dashboard provides administrators with:

- visibility into processed requests,
- request status monitoring,
- asset and room availability tracking,
- and management tools for reviewing approvals.

The frontend communicates with backend API endpoints to retrieve and display live data stored in Supabase.

Backend API (FastAPI)

The FastAPI backend acts as the communication layer between the frontend and database systems.

The backend was intentionally designed to remain lightweight and stateless to simplify scalability and deployment.

Workflow Automation (n8n)

n8n serves as the orchestration engine responsible for:

- fetching unread emails,
- routing requests,
- invoking AI analysis,
- processing approval logic,
- generating automated replies,
- and storing results in the database.

The workflows operate asynchronously, allowing the system to process requests independently of frontend interactions.

AI Processing Layer (Gemini / Groq)

Large language models were integrated to:

- summarize requests,
- classify request types,
- generate recommendations,
- and assist in automated decision making.

AI usage was selectively applied only when necessary to reduce cost and improve efficiency.

Database Layer (Supabase)

Supabase provides:

- centralized storage,
- request tracking,
- asset management,
- and persistent workflow data.

The database stores:

- incoming emails,
- AI summaries,
- approval recommendations,
- request statuses,
- and asset availability information.

Full Documentation Reference

[Insert Technical Documentation / GitHub / Google Docs Link]

Github: <https://github.com/DJpollo/email-agent-reader-Backend>

Scalability Plan:  Scalability Plan

Scalability Implementation Summary:  Scalability Implementation Summary

Specifications Document / PRD:  Specifications Document.docx

Frontend Specs Doc:

<https://github.com/DJpollo/email-agent-reader-Backend/blob/main/Documents/frontend%20specs%20doc.pdf>

Risk Management Plan:

<https://github.com/DJpollo/email-agent-reader-Backend/blob/main/Documents/Risk%20Management%20Plan%20.pdf>

Jira: <https://utdallas-team9-4485-26f.atlassian.net/jira/software/projects/SCRUM/boards/1>

Bug Reports:

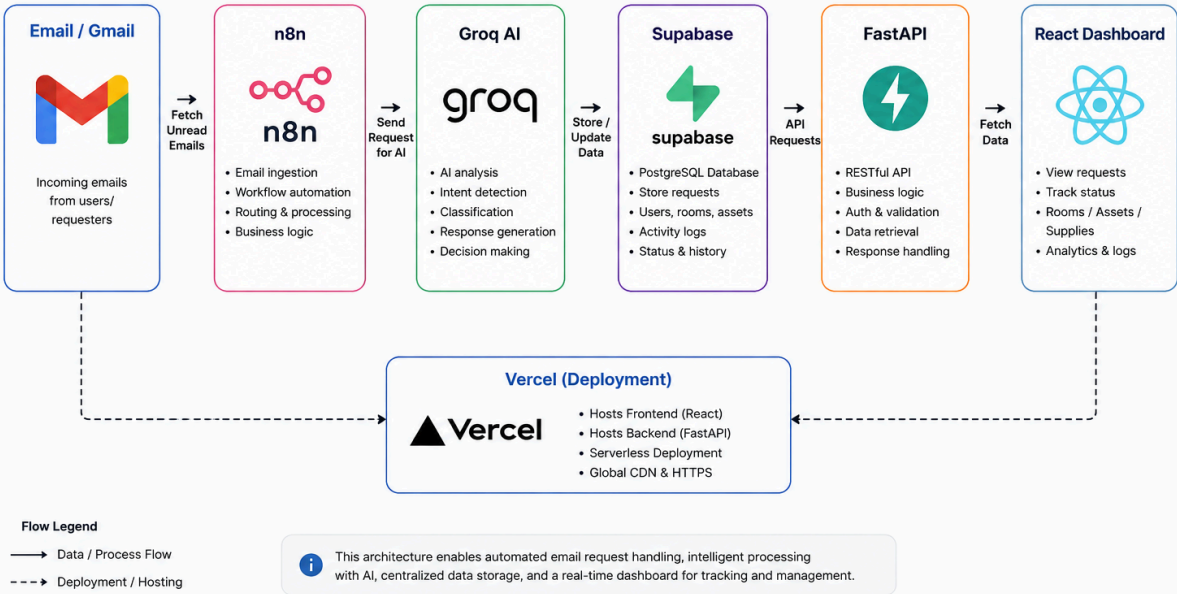
<https://github.com/DJpollo/email-agent-reader-Backend/tree/main/Documents/Bug%20Reports>

3.2 Architecture Diagrams: Visual representations of the system, including high-level architecture diagrams, data flow diagrams, database schemas (ER diagrams), and component interactions.

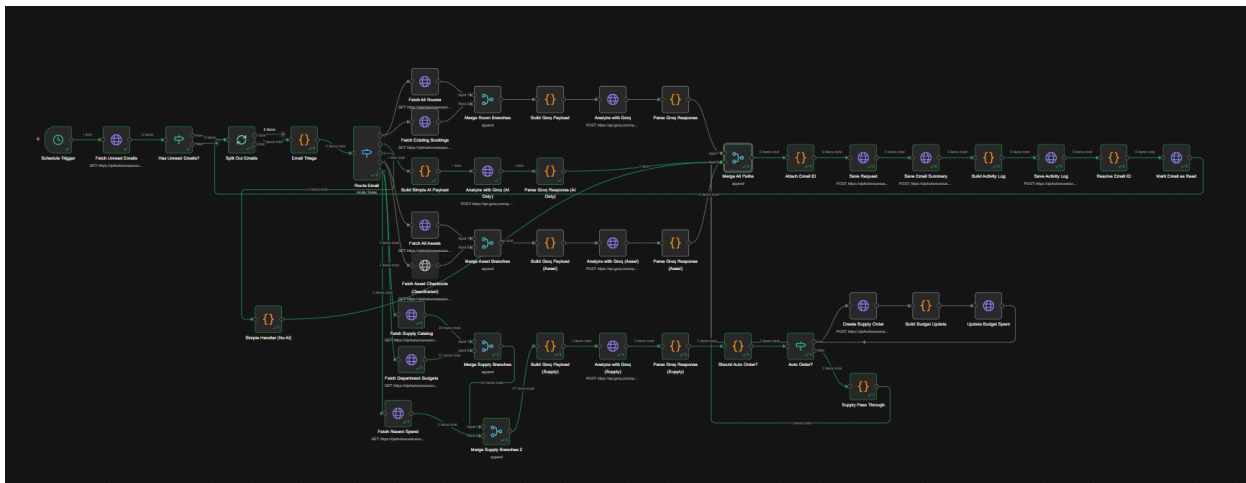
High level architecture diagram:

High-Level Architecture

Email requests are processed through automated workflows and AI, stored in a database, served via an API, and displayed on a web dashboard.



N8n workflow:

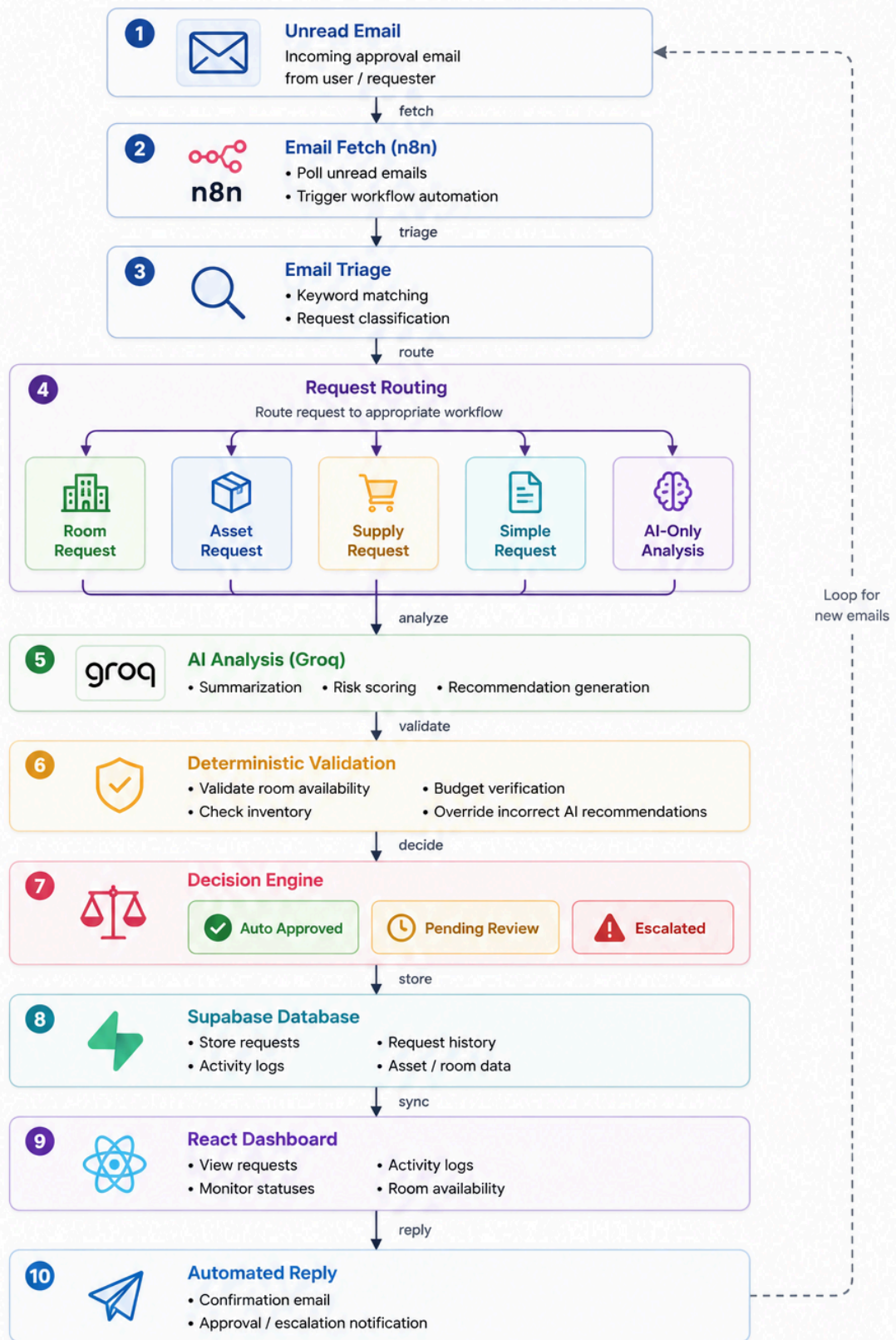


Schemas in separate document because they take up too much space

Another architecture diagram:

Email Processing & Workflow Data Flow

AutoApprove System



3.3 Key Architectural Decisions and Trade-offs: An explanation of the critical design choices made, specifically how scalability was addressed and the trade-offs considered between different technologies or patterns.

One of our main architectural decisions was using n8n instead of directly building everything with the Gmail API. While the Gmail API could technically read and send emails, n8n gave us a visual workflow system where we could connect email ingestion, AI analysis, Supabase updates, and automated replies in one place. This made development faster and easier to debug. The trade-off is that n8n can become harder to manage as workflows grow, so we had to organize the logic carefully and redesign parts of the workflow to support processing multiple emails more efficiently.

We also chose Supabase as our backend database because it provided a hosted PostgreSQL database, built-in APIs, authentication support, and an easy way to store emails, requests, room bookings, assets, and activity logs. This helped us avoid spending too much time setting up database infrastructure from scratch. The trade-off is that we rely on Supabase as a managed platform, but for our project scope, the speed of development and reliability were more important than managing our own database server.

For scalability, we focused on making the system asynchronous and able to handle multiple requests at once. Instead of processing one email manually from start to finish, the workflow can process batches of emails and route them through different paths based on request type. We also switched from Gemini to Groq to improve AI response speed and reduce workflow delays. The main trade-off is that adding concurrency makes the workflow more complex, but it allowed the system to handle more emails in less time while still keeping the approval logic organized and auditable.

Implementation & AI Orchestration

4.1 Technologies Used: A detailed list of the tech stack (e.g., React, Node.js, PostgreSQL), explaining where each technology fits within the application and its specific purpose.

- **React** — Used to build the frontend dashboard and user interface for viewing requests, room availability, activity logs, and asset management features.
- **FastAPI** — Used as the backend API framework to handle application logic, API endpoints, and communication between the frontend, workflows, and database.
- **Supabase (PostgreSQL)** — Used as the hosted database backend for storing emails, requests, room bookings, assets, supply requests, activity logs, and workflow results.
- **n8n** — Used for workflow automation and orchestration, including email ingestion, request routing, AI analysis, automated approvals, and workflow execution.
- **Groq API** — Used for AI inference, including email summarization, request analysis, and risk scoring during workflow processing.
- **Google Gemini API** — Initially used for AI processing before later transitioning to Groq for improved performance and throughput.
- **Vercel** — Used to deploy and host both the frontend and backend services using serverless infrastructure.
- **JavaScript** — Used throughout the frontend and within n8n workflow nodes for custom workflow logic and automation scripting.
- **SQL** — Used for querying and managing Supabase database tables and workflow-related data.
- **GitHub** — Used for version control, collaboration, and managing project code throughout development.
- **Codex and Claude** — Used as AI-assisted development tools for planning, debugging, generating PRD specifications, and accelerating implementation workflows.

4.2 AI Build Strategy: Documentation of the AI engineering techniques used, such as system personas and instruction logic provided to the agents. Provide appropriate links to your prompt library or documentation.

(If there is an AI in your product and it requires a prompt to fail, this is an AI prompt not used during build strategy but rather during the running of the app).

Our AI build strategy focused on structured orchestration rather than using AI in an unrestricted way. The system was designed so that the model acted as an

approval-analysis agent, reading incoming emails and returning structured fields such as category, urgency, risk score, recommendation, and summary. To make the AI output reliable enough for the application, the team combined prompt instructions with backend validation logic. The workflow constrained the model to return JSON only, enforced allowed field values, applied fallback parsing when responses were imperfect, and converted the model's output into application statuses such as auto-approved, pending, or escalated. Documentation for this strategy was maintained in the project workflow files and configuration notes.

4.3 Human-in-the-Loop Interventions: A log of instances where the team had to manually intervene, debug code, override AI-generated logic, or resolve model "hallucinations".

- AI-generated frontend code was often inconsistent and sometimes tried to modify too many parts of the application at once, which made debugging difficult. Because of this, we frequently had to manually review generated code, simplify tasks, and change implementation plans during development.
- During testing, the AI model would occasionally recommend manual review for valid room booking requests even when rooms were clearly available. To solve this, we added deterministic validation logic that overrides the AI recommendation when the workflow already confirms the request is valid.
- Workflow debugging in n8n also required significant manual intervention. Small configuration issues involving Supabase queries, routing logic, or API responses could break multiple parts of the automation pipeline, so workflows had to be repeatedly tested and adjusted by hand.
- We also encountered development and deployment issues involving GitHub, frontend-backend integration, Supabase configuration, and API setup that required manual debugging and troubleshooting beyond AI-generated suggestions.

4.4 Codebase: References to your project's GitHub repository, including links to the main README.md file and other primary documentation.

Github: <https://github.com/DJpollo/email-agent-reader-Backend>

Readme: <https://github.com/DJpollo/email-agent-reader-Backend/blob/main/README.md>

Scalability Plan: [Scalability Plan](#)

Scalability Implementation Summary: [Scalability Implementation Summary](#)

Specifications Document / PRD: [Specifications Document.docx](#)

Frontend Specs Doc:

<https://github.com/DJpollo/email-agent-reader-Backend/blob/main/Documents/frontend%20specs%20doc.pdf>

Risk Management Plan:

<https://github.com/DJpollo/email-agent-reader-Backend/blob/main/Documents/Risk%20Management%20Plan%20.pdf>

System Performance & Validation

5.1 Testing and Validation: An overview of the strategies used to verify the code, including tools like Postman and testing frameworks.

Code validation was mainly done manually through bug testing. By testing inputs through the frontend, we simulated the average users' workflow, be it setting up a request, manually reviewing unsafe requests, etc.. We caught quite a few errors through this method; things like UI bugs, major bugs which showed sensitive data, and more minor bugs that would confuse users were all caught manually. Notable examples, like invisible buttons, auto-booking not working due to an overly-conservative AI, raw database values being leaked, and requesters being able to approve/deny requests were fixed after being discovered through bug tests. For our purposes, this method of testing worked fine, though automation tools could be used in the future for more in-depth stress testing.

5.2 Technical Performance Metrics: Data on system health, such as page load times, API latency, and resource utilization.

During scalability testing, the system processed 33 emails in about 18 seconds using

concurrent workflow execution. We redesigned the workflow to support parallel processing and switched from Gemini to Groq for faster multi-request AI inference and lower latency.

5.3 AI Model Performance (if included): Success rates for AI-generated insights and natural language parsing accuracy scores.

The project did not include a formal benchmark dataset, so no official accuracy percentage was reported for natural language parsing. Instead, model performance was evaluated by whether the AI could consistently generate structured outputs that the system could successfully parse and store. Reliability was improved by using strict JSON output instructions, low-temperature generation, validation against expected categories and labels, and fallback logic for malformed responses. Based on manual testing and end-to-end workflow demonstrations, the model performed well enough for the prototype, though future work should include formal measurements for parsing accuracy, routing accuracy, and failure rate.

Lessons Learned

6.1 Insights Gained: Reflections on key takeaways and the shift from traditional coding to AI-orchestrated engineering.

One of the main insights gained from this project was that AI engineering differs from traditional software development. In addition to writing code, the team had to design prompts, define output structures, validate model responses, and build safeguards around unpredictable AI behavior. This created a shift from simply implementing features to engineering systems that combine AI, automation, backend logic, and user interfaces in a controlled and reliable way. The project showed that AI is most effective when supported by strong validation, structured workflows, and human oversight.

6.2 Lessons from Challenges: A list of technical "walls" encountered (such as AI limitations) and the strategies used to overcome them.

One of the major challenges we encountered during development was learning how to work effectively with AI tools. Though it sped up development, it had its limitations such as inconsistent frontend design, as well as making too many changes at once. As a result, understanding the project structure became difficult and we frequently had to change plans, simplify tasks, and break features into smaller steps.

Another challenge was integrating multiple technologies together, especially n8n workflows and Supabase. Since many parts of the system depended on other parts, even small changes could have huge effects. We utilized iterative testing, modular design, and validation workflow logic before entirely relying on AI outputs.

6.3 Skills Developed: Assessment of how the team's skills in AI engineering, teamwork, etc.

One of the main skills we developed throughout this project was learning how to effectively use AI-assisted development tools to accelerate workflow design, planning, and implementation. While the overall project idea and architecture were created by our team, we used tools such as ChatGPT and Claude to help structure development tasks, generate planning documents, and refine implementation strategies.

A major example of this was creating detailed PRD (Product Requirements Document) specifications that described the system's workflows, database structure, and backend

requirements. These documents helped guide development and allowed us to iterate on features more efficiently during implementation.

In addition to AI-assisted planning, we also gained hands-on experience working directly with technologies such as Supabase, n8n, FastAPI, React, and Vercel. Many parts of the project initially required manual experimentation and debugging, particularly when designing database schemas, configuring automation workflows, and improving workflow scalability.

6.4 Individual Reflection: Brief description about personal insights, challenges and skills developed (Each member must complete this section)

Luis Jaimes

Coming into this, I didn't really have any experience with many of the tools mentioned in our meetings. I've only been working on a few projects on the side and all the class projects, so coming in this and knowing we'd be able to build whatever we wanted was refreshing. Having weekly meetings was also a new thing for me, and I was pleasantly surprised by how everything ran smoothly this whole semester. The only true challenges were the new tools I'd have to learn how to use to be useful in my group. tons of hours trying to get things to work, such as the n8n workflows, getting the backend and front end connected, setting up the supabase, setting up new email accounts, downloading multiple stuff to the laptop, were all very frustrating, but eventually figuring it out. Communication was also a huge challenge in our team, but as long as I got my work done and shared it with my team, then that's all I could do in the situation. Other challenges that I still have problems with are the use of GitHub and putting in the right commands(one command completely wiped my GitHub, so I had to restart). Skills developed mainly focus on the backend since that's what I worked on the most. The use of AI definitely helped, but with the limited tokens in Claude, I had to do things by hand sometimes. Seeing how everything worked via Supabase and n8n, and having the front end show it was all new skills I learned. SQL and Java used in my n8n nodes were also something to get a hold of for sure as well. Overall, I think I learned very valuable skills that I'd definitely be able to use in my future CS career.

Carlos Ortiz III

I never used half of these tools when I started this class. This was my first exposure to a lot of the recent AI tools, but beyond that, I learned how to work with Jira and applied what I learned from my Software Engineering class regarding sprints and the Agile method. I also learned about web design a bit through this course; one of my weakest areas I've found is that I don't quite know a lot about web coding specifically but this course helped me a bit with that. The biggest challenge, much like every other group project I've done, was coordinating progress and parts of our project. There was a lot of overlap at the start and it was a bit hard to understand every moving part but we

persevered through that. I think the biggest thing I learned was keeping up with documentation; that was something that I never really did before and that I had to do a *lot* with this project specifically. I honestly did less coding than I was expecting for my senior year Computer Science project, but I digress. The class was a good insight into a future that wouldn't be too far removed from the average Computer Science-related job.

Shayan Asif

Throughout this semester, I have really developed a larger appreciation for AI-driven development. While I've used AI a lot during Hackathons, I had never used it to the degree where I let it do most of the engineering, whereas I'm mostly responsible for the design and architecture. This was also my first time using a paid AI sub. While my other teammates alternated between Codex and Claude, I mostly used Codex. The main thing I learned was how good AI is for development when you actually give it project specifications and good prompts. However, one major challenge I faced was that sometimes the AI would work too fast and I did not know where to go from there. I learned that proper reviewing and testing of code is very efficient, and I can't just ask AI to do the development without proper planning. Also, Codex wasn't that good at frontend design but I was able to improve that by using Codex skills. I learned skills such as software planning and testing, architecture design, agentic workflows, and CI/CD pipelines.

Adith Gangalakunta

My role in this project was mainly focused on connecting the system and making the full workflow function properly. I worked on integrating the web app with Supabase, setting up authentication and profiles, improving request handling, and helping transition the automation workflow to Different LLMS. One of my biggest takeaways was learning that a project like this depends on how well all the parts work together, not just whether each feature works on its own.

The main challenges came from integration and debugging. I had to deal with issues involving authentication, database structure, permissions, and keeping the frontend, Supabase, and n8n workflow synchronized. These problems took a lot of troubleshooting, but they helped me better understand how backend systems need to be reliable and secure.

Through this project, I developed stronger skills in backend development, full-stack integration, and problem-solving. I became more comfortable with Supabase, authentication, database migrations, workflow automation, and testing changes to make the system more stable. Overall, this project gave me practical experience that I can apply in future software and computer science work.

Hamere Abate

Throughout this semester, I learned a lot about the different technologies needed to build a full-stack application. I've worked in teams before, but not on a project like this where I got to help build a system using AI, automation, databases, and frontend/backend development together. I also worked on backend workflow automation using n8n and API integrations. I learned how workflows, APIs, databases, and AI systems connect and work together in a real-world application. I also worked on scalability in the n8n workflow, so I learned about concepts such as asynchronous processing, AI response handling, and reducing bottlenecks in large request systems.

Some challenges I faced while working on the n8n prototype and Gmail API integration were learning unfamiliar technologies, debugging workflow execution paths in n8n, and working with API connections such as Gmail OAuth. Since many parts of the system were connected together, even small issues could affect multiple components, which made debugging harder. I also learned the importance of communication and asking questions while working in a team environment.

Lastly, I developed skills in workflow automation, backend debugging, AI-assisted engineering, problem solving, API integration, technical documentation, teamwork, and understanding how scalable systems are designed and managed. This project gave me practical experience working on a real-world software engineering project that combined AI, databases, automation, and cloud technologies into one integrated system.

Conclusion & Future Work

7.1 Summary of Key Accomplishments: A final reflection on the project's success in meeting its original objectives.

Overall, our project successfully addressed the problem of automating approval and scheduling workflows through the use of AI-assisted processing and workflow automation. We developed a system capable of filtering simple requests from more complex ones and autonomously handling actions such as room scheduling, asset lending, and approval routing. The platform supports multiple request types and is able to process requests efficiently through our n8n-based automation pipeline and AI analysis workflows.

One of our main accomplishments was building a fully connected system that integrates frontend dashboards, backend APIs, AI processing, workflow orchestration, and cloud-hosted databases into a single working platform. Throughout development, we also improved the scalability of the workflow by redesigning the automation process to support concurrent request handling and faster AI response times.

Although some planned features, such as Google OAuth integration and additional notification systems, were not fully implemented, the overall system met and, in several ways, exceeded the goals outlined in our original project proposal. In addition to building a functional product, the project gave our team valuable experience working with modern development tools, AI-assisted engineering workflows, cloud deployment, and collaborative software development.

7.2 Future Enhancements: A roadmap for future stages, describing how the product could be scaled or improved.

We intend to enhance the system's usability, scalability, and configurability in further phases of development. Strengthening authentication, enhancing secure user data storage, and putting in place more sophisticated role-based access control are some of the main objectives for increasing support for multi-user systems. In order to better accommodate concurrent email processing and AI demands, we also intend to keep improving workflow scalability. In order to make it easier for businesses to adapt workflows to their own requirements, we also wish to introduce reusable workflow templates for typical approval scenarios like accommodation reservations, asset checkouts, and supply requests. Future improvements might potentially include better analytics and monitoring capabilities, as well as interfaces with platforms like

Microsoft Teams for notifications and approval updates.

References

References

n8n Documentation — <https://docs.n8n.io>

Supabase Documentation — <https://supabase.com/docs>

FastAPI Documentation — <https://fastapi.tiangolo.com>

React Documentation — <https://react.dev>

Vercel Documentation — <https://vercel.com/docs>

Groq API Documentation — <https://console.groq.com/docs>

Google Gemini API Documentation — <https://ai.google.dev>

PostgreSQL Documentation — <https://www.postgresql.org/docs>

OpenAI ChatGPT — <https://chat.openai.com>

Anthropic Claude — <https://claude.ai>

v0 by Vercel — <https://v0.dev>

Canva — <https://www.canva.com>

Frontend design and AI-assisted UI generation tools used for interface prototyping and design refinement

Acknowledgements

We would like to thank our professor, Sridhar Alagar, for providing guidance and feedback throughout the development of this project. We also wanted to thank our TA, Adarsh Gella, for their support. Their feedback and recommendations helped us improve a lot in terms of technical and presentation aspects of the project.